

Trabajo Práctico Final - Principios de la Robótica Autónoma (I-402)

Giacometti, Mateo
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
mgiacometti@udesa.edu.ar

Martín Bernal, Tiziano Leví
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
tmartinbernal@udesa.edu.ar

I. INTRODUCCIÓN

A continuación, se presenta el informe del **trabajo práctico final** de la materia *Principios de la Robótica Autónoma (I-402)* perteneciente a la carrera de **Ingeniería en Inteligencia Artificial** de la *Universidad de San Andrés*. Este trabajo práctico, dividido en **dos partes** (en adelante A y B), tiene como objetivo integrar los principales conceptos abordados durante la cursada, aplicándolos en la implementación de un **sistema autónomo de localización, mapeo y navegación** utilizando un robot *TurtleBot3* simulado en el entorno de *Gazebo*.

En la **Parte A**, se desarrolló un sistema de *Simultaneous Localization and Mapping (SLAM)* basado en un *Filtro de Partículas (FastSLAM)*. El objetivo fue que el robot explorara un entorno desconocido tipo laberinto, utilizando datos del sensor LIDAR y odometría para construir un **mapa de ocupación** preciso mientras estimaba simultáneamente su propia posición. Este proceso involucró la configuración del entorno de simulación en *ROS 2 Humble*, la teleoperación del robot para la exploración inicial, y la implementación de algoritmos que manejaran la incertidumbre inherente a los sensores ruidosos. El mapa generado, visualizado y validado en *RViz2*, fue guardado para servir como base en la siguiente etapa.

En la **Parte B**, se implementó un sistema de navegación autónoma que permitió al robot **desplazarse entre puntos arbitrarios dentro del mapa previamente construido**. Este sistema integró módulos de localización probabilística inicial y continua (mediante un *Filtro de Partículas*), planificación de trayectorias en el mapa de ocupación, y control de seguimiento de caminos. La navegación fue diseñada para ser robusta frente a incertidumbre, garantizando un desplazamiento seguro y eficiente hacia los objetivos establecidos.

Ambas partes del trabajo práctico se desarrollaron en un paquete de *ROS 2* autocontenido, utilizando **Python** para los nodos principales. Este informe describe la arquitectura del sistema, las decisiones de diseño tomadas, los resultados obtenidos y un análisis crítico del desempeño del robot en el entorno simulado.

II. PARTE A - GENERACIÓN DE MAPA

Para esta parte del trabajo práctico, como se dijo en la introducción, teníamos como objetivo implementar un sistema de *Simultaneous Localization and Mapping (SLAM)* basado en un *filtro de partículas (FastSLAM)* capaz de explorar un entorno desconocido y construir un mapa de ocupación utilizable para la navegación posterior. Para ello, la cátedra nos proporcionó los siguientes recursos:

- **Modelos del robot:** Archivos *URDF* y *meshes* correspondientes al *TurtleBot3 Burger*, con calibración de parámetros de odometría.
- **Entorno de simulación:** Un mundo tipo laberinto en *Gazebo* a partir del cual se busca generar el mapa de ocupación.
- **Paquete de ROS 2 inicial:** Estructura de directorios con un *nodo* plantilla en *Python* (`python_slam_node.py`), archivos de lanzamiento y la configuración básica de *topic names*.

A continuación, se describen las etapas principales del desarrollo del nodo de generación de mapa y se expondrán los resultados obtenidos.

II-A. Elección de parámetros y criterios

Dentro de la clase `PythonSlamNode`, se nos solicitó definir los siguientes **atributos**:

- `map_resolution`: Define el tamaño de cada celda del mapa en metros. Una resolución más chica da mayor detalle pero requiere más memoria mientras que una muy baja resolución puede perder precisión en entornos angostos. En nuestro caso, establecimos este valor en **0.05 m**, es decir, cada celda representa un **cuadrado de 5 cm × 5 cm**. Este valor fue seleccionado porque garantiza un consumo de recursos computacionales razonable en una amplia variedad de equipos.
- `map_width_meters`: Indica el ancho total del mapa en metros. Establece cuánto espacio físico horizontal cubrirá el mapa. En nuestro caso, establecimos este valor en **5 m** ya que, luego de algunas pruebas, logramos determinar que esta era la dimensión correspondiente al laberinto que debíamos mapear.

- `map_height_meters`: Indica la altura total del mapa en metros. Define el alcance vertical del entorno que se mapea. En nuestro caso, establecimos este valor en **5 m** ya que, luego de algunas pruebas, logramos determinar que esta era la dimensión correspondiente al laberinto que debíamos mapear.
- `num_particles`: Es la cantidad de partículas del filtro de partículas. Más partículas permiten mejor estimación, pero consumen más recursos. Por otro lado, menos partículas consume menos recursos y aumenta la velocidad, pero a cambio se puede perder precisión o convergencia. En nuestro caso, establecimos este valor en **10** ya que garantiza un consumo de recursos computacionales razonable en una amplia variedad de equipos.
- `log_odds_occupied`: Es un valor que representa cuánto confiamos en que una celda está ocupada (es decir, que hay una pared u obstáculo) cuando el sensor láser detecta algo en ese lugar. Luego de algunas pruebas, definimos este valor de probabilidad en 0.9 (0.95 aproximadamente en *log-odds*).
- `log_odds_free`: Es un valor que representa cuánto confiamos en que una celda está libre (sin obstáculos) cuando el láser pasa sin detectar nada. Luego de algunas pruebas, definimos este valor de probabilidad en 0.40 (-0.18 aproximadamente en *log-odds*).
- `max_range`: Es un valor que determina la distancia máxima que se considera válida para las mediciones del escáner láser al actualizar el mapa de ocupación. El valor seleccionado fue de **2 m**.

II-B. Implementación del método `scan_callback(self, msg: LaserScan)`

Dentro del sistema del nodo `PythonSlamNode`, el método `scan_callback` se encarga de constituir el núcleo operativo del sistema **SLAM** basado en *filtro de partículas*. Cada vez que se recibe un nuevo mensaje de escaneo láser, este método ejecuta una secuencia de pasos que permiten actualizar tanto la estimación de la pose del robot como el mapa del entorno. En primer lugar, se calcula el movimiento relativo del robot a partir de la odometría, y se aplica dicho desplazamiento a cada partícula del filtro, incorporando ruido para simular la incertidumbre inherente al movimiento. A continuación, se evalúa la consistencia de cada partícula comparando su mapa interno con las nuevas mediciones del escáner láser, ajustando su peso en función del grado de coincidencia. Posteriormente, se realiza un proceso de remuestreo que favorece las partículas más verosímiles y elimina las menos representativas, lo cual mantiene la eficiencia y estabilidad del sistema. Con las partículas actualizadas, se estima la nueva pose del robot mediante un promedio ponderado y se actualizan los mapas individuales en base a las nuevas observaciones. Finalmente, se transmite la transformación entre los marcos de referencia `map` y `odom`, lo que permite visualizar correctamente la posición del robot dentro del mapa en herramientas como **RViz**. Este proceso cierra el ciclo de percepción y localización del sistema **SLAM** en tiempo real.

Al implementar este método, nos enfrentamos a diversas dificultades y decisiones importantes. Una de las principales fue la correcta interpretación y manejo de la odometría. Al recibir la orientación del robot en formato de cuaterniones, fue necesario desarrollar un método propio para convertirla a ángulos de Euler: `quaternion_to_euler_yaw(x: float, y: float, z: float, w: float)`, ya que no logramos hacer funcionar correctamente las librerías externas que ofrecen esta funcionalidad.

Otro aspecto crítico fue la necesidad de normalizar los ángulos en distintas etapas del cálculo, especialmente al estimar rotaciones relativas y al actualizar la orientación de las partículas. Esto permitió evitar errores acumulativos y discontinuidades debidas a saltos angulares. Para resolverlo, definimos el método `normalize_angle(angle: float)`, cuya función es mantener cualquier ángulo dentro del rango $[-\pi, \pi]$.

También nos enfrentamos a la decisión de cómo representar la pose estimada del robot: analizamos tanto el uso del promedio ponderado de todas las partículas como la alternativa de seleccionar directamente la mejor partícula (es decir, la de mayor peso). Finalmente, nos decantamos por la opción del promedio ponderado de partículas.

Por otra parte, otro desafío importante fue la selección adecuada de los parámetros de ruido en el modelo de movimiento. Estos coeficientes, que controlan la dispersión aplicada a las partículas durante la etapa de predicción, influyen directamente en la estabilidad, precisión y capacidad de adaptación del filtro de partículas frente a la incertidumbre del entorno. Su ajuste requirió un proceso iterativo de prueba y error. Tras diversas simulaciones, determinamos que una desviación estándar de **0.05 m** para las componentes X e Y , y de **0.1 rad** para la orientación θ , representan razonablemente la incertidumbre típica de la odometría en condiciones reales.

No obstante, el error que más tiempo nos demandó fue el relacionado con la actualización de la posición de las partículas. Inicialmente, implementamos un modelo de movimiento basado en lo visto en clase, aplicado anteriormente en el trabajo práctico del filtro de partículas y reutilizado en la *Parte B* de este trabajo. Sin embargo, al aplicarlo en este contexto observamos que la estimación de la posición del robot en el marco del mapa se degradaba progresivamente a medida que avanzaba por el entorno.

Tras aislar y probar el modelo de movimiento en un archivo aparte, pudimos observar cómo las partículas incrementaban su dispersión con cada iteración. Esto se debía a que sus nuevas posiciones se generaban en torno a la posición anterior de cada partícula, en lugar de centrarse en una fuente de movimiento común como la odometría. Como resultado, el filtro intentaba estimar la posición del robot promediando múltiples estimaciones incorrectas, lo que provocaba una clara divergencia en la localización.

II-C. Implementación del método `compute_weight(self, particle: Particle, scan_msg: LaserScan)`

El método `compute_weight` cumple una función fundamental dentro del ciclo del filtro de partículas: estimar cuán probable es que una partícula represente correctamente la posición real del robot en base a las mediciones actuales del escáner láser. Cada partícula mantiene su propia hipótesis de la pose del robot y su propio mapa del entorno. El método compara las mediciones actuales con el contenido del mapa asociado a cada partícula y asigna un peso que cuantifica esa coherencia. Cuanto mayor es el peso, más consistente resulta la partícula con la percepción actual del entorno. Este valor será utilizado en la fase de resamplio para conservar las partículas más representativas y descartar las menos probables, guiando así la convergencia del sistema hacia una estimación precisa de la posición del robot.

El método de asignación de pesos elegido es bastante simple pero efectivo: compara las mediciones actuales del escáner láser con el mapa construido por cada partícula para evaluar qué tan bien esa partícula representa la posible posición real del robot. Para cada rayo del escáner, se calcula su punto de impacto en el mapa según la pose de la partícula, y se verifica si ese punto coincide con una celda que la partícula considera ocupada. Si hay coincidencia, se incrementa el peso de esa partícula. En otras palabras, se recompensa a las partículas cuyas predicciones sobre el entorno coinciden con las observaciones actuales. Este enfoque permite seleccionar, durante el resamplio, las partículas más consistentes con la realidad percibida, guiando la convergencia del filtro de partículas hacia estimaciones de pose más precisas.

II-D. Implementación del método `resample_particles(self, particles: list[Particle])`

El método `resample_particles` implementa la etapa de remuestreo del filtro de partículas, cuya función principal es concentrar el conjunto de partículas alrededor de las hipótesis más probables. Una vez que cada partícula ha recibido un peso representando cuán bien explica las observaciones sensoriales actuales, este método selecciona un nuevo conjunto de partículas a partir del actual, privilegiando aquellas con mayor peso. En términos prácticos, partículas con alta probabilidad serán copiadas más veces, mientras que aquellas con peso bajo tenderán a desaparecer. Esto permite que el sistema centre su capacidad de estimación en regiones del espacio donde es más probable que se encuentre el robot, descartando hipótesis inconsistentes. Cada nueva partícula conserva la pose y el mapa de la partícula original de la que fue copiada, pero con peso uniforme, reiniciando así el ciclo de estimación para la siguiente iteración. Esta etapa es clave para la convergencia y estabilidad del filtro, ya que mantiene un balance entre explotación (reforzar buenas estimaciones) y exploración (conservar diversidad ante ambigüedad o ruido).

Esta sección en particular no propuso demasiada dificultad ya que simplemente consistió en utilizar y adaptar nuestra implementación de método de resamplio **Muestreo Estocástico Universal (SUS)** realizado para el trabajo práctico 3 de la materia. Seleccionamos este método de resamplio ya que el mismo garantiza una cobertura eficiente del espacio de partículas y evita la pérdida prematura de diversidad.

II-E. Implementación del método `update_map(self, particle: Particle, scan_msg: LaserScan)`

El método `update_map` se encarga de actualizar el mapa de ocupación asociado a una partícula utilizando las nuevas mediciones del escáner láser. Para cada rayo del escáner, el método calcula la trayectoria desde la posición actual del robot (según la partícula) hasta el punto donde el rayo impacta o alcanza su rango máximo. A lo largo de esa trayectoria se marcan las celdas por las que pasa como libres, y si el rayo detecta un obstáculo dentro de su rango útil, se incrementa el valor de ocupación de la celda correspondiente al punto de impacto. Este proceso se realiza modificando los valores de *log-odds* en el mapa, lo que permite representar de forma probabilística el grado de certeza sobre la ocupación de cada celda. Para trazar la línea entre el origen del rayo y su destino, se utiliza el **algoritmo de Bresenham** (mediante un método proporcionado por la cátedra), que determina qué celdas atraviesa un rayo en una grilla discreta. Esta actualización permite que cada partícula construya su propia versión del mapa del entorno, la cual luego se utilizará tanto para calcular su peso como para construir el mapa global publicado.

Una de las principales dificultades al implementar el método `update_map` estuvo relacionada con el cálculo correcto de los ángulos de cada rayo láser y su proyección desde el sistema de coordenadas local del robot hacia el sistema global del mapa. Fue necesario trabajar con transformaciones entre ternas de referencia, combinando rotaciones y traslaciones para convertir cada medición desde el marco del escáner (que asume que el robot está en el origen) hacia coordenadas globales, tomando como referencia la pose de cada partícula. Esto implicó aplicar rotaciones con el ángulo de orientación de la partícula y sumarlas a su posición estimada en el mapa. Estos aspectos geométricos fueron críticos para que los puntos de impacto de los rayos se ubiquen correctamente sobre el mapa de celdas, ya que pequeños errores angulares podían resultar en celdas mal actualizadas o mapas inconsistentes. La precisión en estas transformaciones fue clave para lograr una actualización confiable del entorno.

II-F. Implementación del método `publish_map(self)`

El método `publish_map` se encarga de generar y publicar un mapa de ocupación global en el formato específico soportado por **ROS** (*nav_msgs/OccupancyGrid*) a partir de los mapas individuales mantenidos por las partículas del filtro. Se selecciona

una partícula de referencia (en este caso, aquella que cuente con el mayor valor de peso asociado) y se utiliza su mapa para construir la representación final del entorno. Para ello, se recorre la matriz de *log-odds* de dicha partícula y se convierte cada valor a una probabilidad de ocupación. Además, se configuran los metadatos del mensaje como la resolución, el tamaño de la grilla y la posición del origen del mapa en coordenadas globales. Finalmente, el mensaje es publicado en el tópic `/map` para que otros nodos del sistema (por ejemplo, visualizadores como **RViz**) puedan acceder al mapa generado.

En el momento de implementar este método, nos encontramos con algunas dificultades producto de falta de conocimiento de **ROS** y problemas derivados de implementaciones iniciales incorrectas. Referido a lo mencionado en primer lugar, se nos presentó el desafío de tener que entender de qué forma debíamos realizar la publicación del mapa para que el mismo fuera interpretado correctamente por **ROS**. Esto implicó aprender cómo configurar correctamente el mensaje `OccupancyGrid`, definir adecuadamente los metadatos como la resolución, dimensiones del mapa y la posición del origen, y comprender el formato en que debían enviarse los datos de ocupación. Por otro lado, también surgieron errores relacionados con la conversión de valores de *log-odds* a probabilidades, y con la forma de representar las celdas desconocidas. Por una parte, una de nuestras primeras implementaciones nos generó complicaciones al ser computacionalmente exigente de forma innecesaria. Lo que hacíamos era recorrer mediante un doble `for` anidado cada celda del mapa de la partícula seleccionada para determinar el nivel de ocupación de acuerdo al valor de *log-odd* almacenado. Al pasar a realizar las actualizaciones de forma vectorizada, obtuvimos una mejora considerable de rendimiento. Otro problema se relacionaba a la forma en la que definíamos las probabilidades de ocupación del mapa a publicar. En un principio, lo establecimos de forma que cada celda con *log-odd* menor a `log_odds_free` pasaba a figurar como desocupada (valor 0 dentro de la celda correspondiente en el mapa a publicar) y aquellas con *log-odd* mayor a `log_odds_occupied` pasaban a figurar como ocupadas (valor 100 dentro de la celda correspondiente en el mapa a publicar). De esta forma, obteníamos un mapa con muchas celdas cercanas al robot que figuraban como desconocidas (valor -1 dentro de la celda correspondiente en el mapa a publicar) a la vez que el nivel de ocupación de cada celda variaba mucho con cada actualización. Debido a esto, terminamos definiendo el criterio final utilizado: transformar todas las *log-odds* a **odds** para luego multiplicarlas por 100 y publicar este mapa con el formato adecuado para **ROS**.

II-G. Implementación del método `broadcast_map_to_odom(self)`

El método `broadcast_map_to_odom` se encarga de publicar la transformación entre los marcos de referencia `map` y `odom`, necesaria para alinear la estimación global del SLAM con la odometría local del robot. Esta transformación permite que otros nodos del sistema puedan conocer cómo se relaciona la pose estimada global con el sistema de referencia odométrico, que tiende a acumular error con el tiempo. Para calcular esta transformación, se toma la diferencia entre la pose del robot estimada por el filtro de partículas en el marco `map` y la pose informada por la odometría en el marco `odom`. Se utiliza esta diferencia para construir una transformación rígida compuesta por una traslación (desplazamiento en X e Y) y una rotación (calculada a partir del ángulo entre orientaciones), expresada como un cuaternión. El resultado se empaqueta en un mensaje `TransformStamped` y se publica a través de un `TransformBroadcaster`. De este modo, se nos permite mantener actualizada la relación entre el marco odométrico y el mapa global, asegurando la coherencia espacial en todo el sistema implementado.

La elaboración del método presentó desafíos técnicos importantes, especialmente relacionados con el manejo correcto de transformaciones entre marcos de referencia. Uno de los principales problemas fue comprender cómo calcular la transformación inversa entre las poses estimadas en el marco `map` y las reportadas en el marco `odom`. Esto requirió un entendimiento preciso de las transformaciones rígidas en el plano, tanto a nivel matemático como en su representación dentro del sistema de mensajes de **ROS**. Además, surgieron dudas sobre si era necesario utilizar matrices de transformación homogénea o si bastaba con aplicar una rotación seguida de una traslación, lo cual llevó a cometer errores iniciales en el signo y en el orden de las operaciones. También fue necesario convertir correctamente la rotación angular a un cuaternión, cuidando de normalizar el ángulo y evitar inconsistencias que pudieran producir saltos o inestabilidad en la visualización del robot en **RViz**.

II-H. Resultados

A la hora de ejecutar el código para realizar un proceso de mapeo, nos vamos a encontrar algo como se muestra en 1:

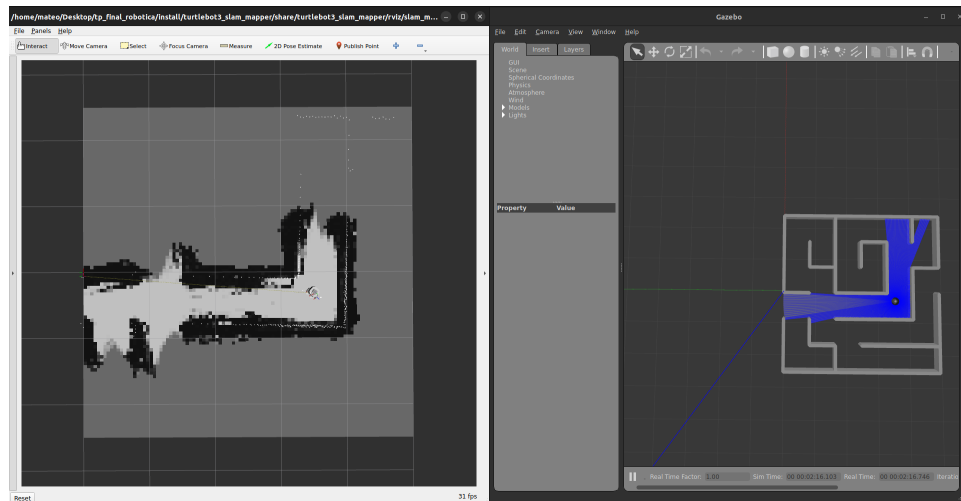


Figura 1: Proceso de mapeo. Despliegue de las aplicaciones *Rviz2* y *Gazebo*.

Cabe aclarar que tiene que desplegar una consola adicional para así poder ejecutar los comandos correspondientes para el manejo del robot.

Una vez finalizado el proceso, podemos obtener un resultado como se ve en 2:



Figura 2: Mapa resultante de una secuencia de mapeo.

Finalmente, presentamos un video de demostración mostrando el correcto funcionamiento de esta etapa del trabajo.

III. PARTE B - NAVEGACIÓN AUTÓNOMA CON LOCALIZACIÓN Y SEGUIMIENTO DE TRAYECTORIA

En esta etapa del trabajo práctico, como también fue mencionado en la introducción, el objetivo consistió en implementar un sistema de localización probabilística y navegación autónoma para un robot móvil, integrando localización con planificación de trayectoria y control de seguimiento de camino, incluyendo además evasión de obstáculos.

Para introducir el tema, los algoritmos que se utilizaron para cada parte del trabajo son

- Para localización: filtro de partículas (AMCL)
- Para planificación de trayectoria: A*
- Para seguimiento de camino: Pure pursuit
- Para evasión de obstáculos: heurística propia

Para ello, la cátedra facilitó un **Paquete de ROS 2 inicial**, es decir, una estructura de directorios con un *nodo* plantilla en *Python* (`amcl_node.py`), archivos de lanzamiento y la configuración básica de *topic names*.

A continuación, se describen las etapas principales del desarrollo del nodo de localización y seguimiento de trayectorias y se expondrán los resultados obtenidos.

III-A. Algoritmos implementados

III-A1. Localización con Filtro de Partículas (AMCL): Para localizar al robot dentro del mapa utilizamos una implementación personalizada del algoritmo AMCL (Adaptive Monte Carlo Localization), un filtro de partículas que estima la pose (x, y, θ) del robot mediante una distribución de hipótesis mantenidas a lo largo del tiempo. Este enfoque combina cuatro componentes principales:

- un modelo de movimiento basado en la odometría del robot,
- un modelo de medición que compara observaciones láser con el mapa
- un paso de resampleo
- una etapa final de estimación de la pose a partir de un promedio ponderado de las partículas.

Al iniciar el nodo, si no se recibe una pose inicial, se ejecuta un método de inicialización aleatoria (`initialize_particles_randomly`) que distribuye partículas por todo el mapa. Este enfoque, aunque completo, sufre de un problema conocido como el "problema del robot secuestrado", donde el robot puede tardar mucho en converger a su verdadera posición (o no hacerlo) si la nube de partículas comienza muy alejada de ella y no hay suficientes partículas. Para mitigar esto, también se implementó una función para inicializar partículas alrededor de una pose provista por el usuario, logrando una convergencia más rápida y precisa.

III-A1a. Modelo de movimiento: predice cómo se desplaza cada partícula cuando el robot se mueve, teniendo en cuenta la odometría entre dos lecturas consecutivas. Para ello, se calcula primero el desplazamiento relativo del robot entre dos poses odométricas. Para eso se usan 3 ecuaciones, una para la traslación total entre posiciones, otra para la primera rotación necesaria para orientar el robot hacia el desplazamiento y otra para la segunda rotación que completa el movimiento:

$$\Delta_{trans} = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2} \quad \Delta_{rot1} = \text{atan2}(y_1 - y_0, x_1 - x_0) - \theta_0 \quad \Delta_{rot2} = (\theta_1 - \theta_0) - \Delta_{rot1}$$

Donde (x_0, y_0, θ_0) y (x_1, y_1, θ_1) son las poses del robot antes y después del desplazamiento.

Para cada partícula i , estos incrementos se aplican con ruido gaussiano, modelando incertidumbre en la odometría. Los ángulos y la traslación estimados para la partícula son:

$$\hat{\Delta}_{rot1} = \Delta_{rot1} + \mathcal{N}(0, \alpha_1 |\Delta_{rot1}| + \alpha_2 \Delta_{trans}) \quad \hat{\Delta}_{trans} = \Delta_{trans} + \mathcal{N}(0, \alpha_3 \Delta_{trans} + \alpha_4 (|\Delta_{rot1}| + |\Delta_{rot2}|))$$

$$\hat{\Delta}_{rot2} = \Delta_{rot2} + \mathcal{N}(0, \alpha_1 |\Delta_{rot2}| + \alpha_2 \Delta_{trans})$$

donde $\mathcal{N}(\mu, \sigma)$ indica una variable aleatoria gaussiana con media μ y desviación estándar σ . Los coeficientes $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ controlan la magnitud del ruido y fueron elegidos en mi implementación con un valor de 0.1 cada uno.

Finalmente, cada partícula se actualiza como:

$$x_{\text{nuevo}} = x + \hat{\Delta}_{trans} \cdot \cos(\theta + \hat{\Delta}_{rot1}) \quad y_{\text{nuevo}} = y + \hat{\Delta}_{trans} \cdot \sin(\theta + \hat{\Delta}_{rot1}) \quad \theta_{\text{nuevo}} = \theta + \hat{\Delta}_{rot1} + \hat{\Delta}_{rot2}$$

Para hacer esta parte del trabajo nos focalizamos en revisar uno de los trabajos prácticos realizados durante la cursada, donde se utilizaba un modelo de movimiento y lo adaptamos al presente trabajo.

III-A1b. Modelo de Medición: Para estimar qué tan probable es que cada partícula represente la verdadera posición del robot, implementamos un modelo de medición basado en raycasting. La idea es comparar las distancias medidas por el láser con las distancias que se esperaría leer en el mapa si el robot estuviera en la posición de cada partícula.

En nuestro caso, usamos hasta 60 rayos distribuidos a lo largo del campo de visión del láser. Para cada partícula, hicimos un raycasting sobre el mapa de ocupación. Si el rayo golpea una celda ocupada, calculamos la distancia z_{expected} a ese obstáculo. Luego comparamos esta distancia con la medición real z_{real} usando una mezcla probabilística:

$$p(z_{\text{real}}|x) = z_{\text{hit}} \cdot \mathcal{N}(z_{\text{real}}; z_{\text{expected}}, \sigma) + z_{\text{rand}} \cdot \frac{1}{z_{\text{max}}}$$

donde $\mathcal{N}(\cdot; \mu, \sigma)$ es la densidad de una normal centrada en μ , y z_{hit} y z_{rand} son pesos que suman 1. Utilizamos $z_{\text{hit}} = 0,9$ y $z_{\text{rand}} = 0,1$. El factor z_{rand} modela la posibilidad de lecturas aleatorias (ruido del sensor o superficies reflectantes). Multiplicamos las probabilidades de todos los rayos para obtener el peso final de cada partícula. Para evitar pesos cero, sumamos siempre un pequeño valor 10^{-6} .

III-A1c. Resampleo y Estimación de la Pose: Después de calcular los pesos de cada partícula, realizamos un resampleo sistemático para conservar solo aquellas partículas con mayor probabilidad. Esto permite concentrar las hipótesis en regiones del mapa donde es más probable que se encuentre el robot.

Finalmente, estimamos la pose del robot como el promedio ponderado de todas las partículas:

$$\hat{x} = \sum_i w_i \cdot x_i \quad y \quad \hat{y} = \sum_i w_i \cdot y_i \quad \hat{\theta} = \text{atan2} \left(\sum_i w_i \sin \theta_i, \sum_i w_i \cos \theta_i \right)$$

Uno de los desafíos que se nos presentó en esta parte fue el poder establecer un criterio entre velocidad de cómputo y performance. En un primer momento probamos con 1000 partículas y sin límites de rayos del LIDAR y lo que ocurrió es que constantemente ROS nos mostraba por consola un mensaje como este: "[rviz2-7] [INFO] [1750733196.875396160] [rviz2]: Message Filter dropping message: frame 'base_scan' at time 0.375 for reason 'discarding message because the queue is full'"

Luego de investigar y ver cuál podía ser el problema, descubrimos que todo el proceso de localización estaba tardando mucho más que el tiempo de llamada de la función `timer_callback`, que es la que engloba toda la lógica de la máquina de estados del programa y que estaba configurada para llamarse cada 0.1 segundos. Al identificar que la mayor cantidad de cómputo se estaba dedicando al ray casting (ya que implica recorrer píxel por píxel en el mapa para cada haz láser de cada partícula), consideramos tomar una parte reducida de los rayos como mencionamos anteriormente (60) y de reducir las partículas a 200. Aún así, cada paso de `timer_callback` estaba tomando 0.3 segundos en promedio, por lo que se decidió aumentar el tiempo de llamada de `timer_callback` de 0.1 a 0.5 segundos. Eso dio margen para que se pudiera hacer bien la localización y se llegara a devolver la pose estimada en un tiempo acorde. A su vez, para que no se solapen las llamadas y las respuestas de los callbacks puedan devolverse en un orden incorrecto, organizamos el código de manera que si llega una consulta y todavía no se resolvió la anterior, esa nueva llamada no devuelve nada.

Otra de las situaciones que descubrimos es que la interfaz de RViz2, mediante Robostack, presenta un leve delay al mostrar estimaciones de posición generadas, lo que hizo que dudemos si había un problema en la estimación o en la visualización. Lo descubrimos mostrando logs en consola y verificando que las estimaciones se realizaban a tiempo.

III-A2. Planeamiento con A:* Para el planeamiento de trayectorias implementamos el algoritmo A*, al cual le pasamos un grid inflado del mapa. Inflar las paredes nos permite generar caminos que mantengan al robot a una distancia prudente de los obstáculos, mejorando la navegabilidad. Sin embargo, observamos que si el inflado es excesivo, puede impedir encontrar un camino en zonas angostas o si el objetivo está muy cerca de una pared. Por eso, fue necesario ajustar el parámetro de inflado para lograr un equilibrio entre seguridad y factibilidad del camino y nos quedamos con un valor de `safety_margin_cells` de 5.

III-A3. Navegación con Pure Pursuit: Para la navegación implementamos el algoritmo Pure Pursuit, un método muy utilizado en robótica móvil para seguir trayectorias suaves. La idea básica consiste en seleccionar un punto objetivo (lookahead point) a cierta distancia hacia adelante en el camino, y calcular la curvatura necesaria para dirigir al robot hacia ese punto. Aunque Pure Pursuit es un algoritmo sencillo, en la práctica encontramos varios desafíos al aplicarlo. En particular, tuvimos que modificar el método de búsqueda del punto objetivo para evitar que el robot se desviara excesivamente o se quedara “enganchado” en puntos del camino.

III-A3a. Búsqueda del punto objetivo: Nuestra función `search_target_index` parte de la posición y orientación actual del robot y recorre el camino planeado para encontrar el punto adecuado a seguir. Inicialmente, buscamos el punto más cercano al robot, pero para evitar que el robot retroceda, limitamos la búsqueda a unos pocos puntos hacia atrás (usamos una “ventana de retroceso” de hasta 5 puntos) para que no vuelva sobre sí mismo si se desvía levemente.

Una de las mejoras importantes fue considerar no solo la distancia, sino también la orientación. Para esto, transformamos los puntos del camino al marco local del robot y buscamos puntos que estén por delante (con coordenada `local_x` positiva) y dentro de un radio de lookahead distance. Si hay varios puntos dentro de esa zona, elegimos el más lejano, lo cual nos permitió generar trayectorias más suaves y evitar movimientos zigzagueantes. Si no se encuentra ningún punto dentro del radio, seleccionamos el primero que esté más adelante fuera del círculo, asegurándonos de que el robot no intente volver atrás. También implementamos un chequeo de cercanía al objetivo final para declarar el objetivo alcanzado si el robot se encuentra dentro de una tolerancia predefinida.

Estas modificaciones fueron clave para evitar que el robot girara sobre sí mismo o se desplazara en direcciones no deseadas, sobre todo al salir de situaciones de evitación de obstáculos, donde podía quedar desalineado respecto del camino.

Además, implementamos una estrategia para evitar que el robot vuelva a puntos peligrosos tras haber evadido obstáculos. Al salir del estado de evitación, adelantamos el índice del target point algunos pasos (típicamente cinco), siempre verificando que el nuevo punto esté suficientemente lejos del robot y orientado hacia adelante. Esto resultó crucial para que el robot no se metiera nuevamente en el mismo obstáculo que acababa de esquivar.

III-A3b. Controlador: En nuestro nodo, el método `pure_pursuit_control` es el encargado de generar los comandos de velocidad lineal y angular que siguen el camino planeado. A partir de la pose estimada del robot, se calcula el ángulo al objetivo y la curvatura necesaria para alcanzarlo.

Para mejorar el comportamiento respecto a la implementación básica, realizamos varias modificaciones:

- **Orientación inicial:** Cuando nos setean un nuevo objetivo, le indicamos al robot que se oriente hacia donde apunta el primer `target_point`. Eso nos evita el problema de que el robot esté orientado muy diferente hacia donde tiene que arrancar a ir. Si está muy mal orientado, probablemente el robot deba realizar un arco grande de trayectoria para poder reubicarse, que en espacios reducidos puede llegar a ser causa de choque.
- **Chequeo del ángulo al objetivo:** si el ángulo hacia el punto objetivo es demasiado grande (mayor a 90°), interpretamos que el robot está mirando en la dirección opuesta al camino. En ese caso, en lugar de avanzar, giramos en el lugar para reorientarnos. Esto evitó muchos problemas en los que el robot comenzaba a retroceder o a girar en círculos sin sentido.
- **Cálculo de curvatura:** utilizamos la ecuación $\kappa = \frac{2 \cdot y_{local}}{L_f^2}$, donde y_{local} es la distancia lateral al punto objetivo en el marco local y L_f es la lookahead distance. Esto nos permitió adaptar el radio de giro del robot a cuán desplazado esté respecto al camino.
- **Ajuste dinámico de la velocidad lineal:** una mejora clave fue reducir la velocidad lineal proporcionalmente a la curvatura. Cuanto más cerrada sea la curva, menor velocidad permitimos, para evitar maniobras bruscas y mejorar la estabilidad del seguimiento. Esto lo hicimos con una función que limita la velocidad máxima en función de la magnitud de la curvatura.
- **Limitación de velocidad angular:** para evitar giros excesivamente rápidos que provocaran inestabilidad, restringimos el rango máximo de ω .
- **Evitar giros extremos:** Además del chequeo de $\pm 90^\circ$, incorporamos una condición especial para ángulos muy grandes (por ejemplo $\pm 150^\circ$), donde en lugar de girar bruscamente el robot gira sobre su eje lentamente hasta reorientarse. Esto previno situaciones donde el robot podía girar demasiado rápido y perder estabilidad.

Otro de los mayores desafíos fue el de encontrar un valor óptimo para la lookahead distance. Si tomábamos un valor demasiado chico (por ejemplo, 0.2), el robot comenzaba a zigzaguear constantemente, lo que generaba que la distancia recorrida no vaya aumentando considerablemente y que por el contrario el robot se quede girando en zonas haciendo figuras como círculos o "8"s. Por otro lado, cuando tomábamos un valor muy grande (por ejemplo, 1), el robot encontraba puntos que estaban en zonas del laberinto muy diferentes como para poder llegar a ellas mediante un solo trazado de arco.

Todas estas modificaciones surgieron como respuesta a pruebas en las que observamos comportamientos inestables: el robot solía pegarse demasiado a las paredes, girar sobre sí mismo o retroceder en lugar de avanzar. Iteramos sobre estas funciones para robustecer el seguimiento, lograr trayectorias más suaves y mantener el robot avanzando siempre hacia adelante.

III-A4. Evasión de obstáculos: Para evitar colisiones con objetos no previstos en el mapa (como obstáculos nuevos o paredes cercanas), implementamos un algoritmo sencillo de evasión basado en el análisis de las lecturas del LiDAR.

La lógica se basa en dividir el campo visual del robot en tres zonas: izquierda, centro y derecha. Analizamos las distancias mínimas medidas en cada sector para detectar posibles obstáculos. Si detectamos algo demasiado cerca en la zona central, el robot interrumpe la navegación y pasa a un estado de evasión, girando sobre su eje para intentar liberarse.

Aunque el planteo inicial era simple, en la práctica encontramos varios desafíos que nos obligaron a mejorar el comportamiento:

- **Dirección de giro inconsistente:** Al principio, ante un obstáculo de frente, el robot elegía girar simplemente hacia el lado más "libre" (es decir, hacia donde el LiDAR midiera mayor distancia). Sin embargo, esto provocaba que a veces girara hacia el lado opuesto al objetivo real. Por ejemplo, si debía esquivar un objeto a la derecha, giraba a la izquierda, pero luego se reorientaba bruscamente hacia la derecha, volviendo a acercarse al obstáculo que acababa de evitar. Para resolverlo, incorporamos la idea de analizar la dirección del próximo punto objetivo (`target point`) del path y priorizar girar hacia ese lado. Esto hizo mucho más consistente el sentido de la maniobra de evasión.
- **Cambios de dirección abruptos:** Otro problema que observamos era que, si el robot aún veía el obstáculo de frente después de varios giros pequeños, volvía a cambiar de dirección de giro en cada iteración. Eso generaba movimientos erráticos y "zigzagados" sin salir nunca del bloqueo. Para solucionarlo, definimos una variable interna que guarda el sentido elegido de giro (izquierda o derecha) y lo mantiene constante mientras dure la maniobra de evasión. Solo una vez

despejado el obstáculo, se borra ese dato y se permite volver a decidir la dirección ante un nuevo bloqueo.

- **Reenganche al path demasiado cerca del obstáculo:** Otro aprendizaje importante fue que, tras evadir un obstáculo, el robot volvía a seguir el mismo punto objetivo que estaba demasiado cerca del obstáculo. Esto lo hacía volver a chocar o entrar nuevamente en evasión. Para evitarlo, implementamos una estrategia que, al regresar al estado de navegación, fuerza al robot a “saltar” unos puntos hacia adelante en el path (por ejemplo, 5 puntos) para retomar la trayectoria en una zona más segura y alejada del obstáculo.
- **Velocidades excesivas en evasión:** Al inicio, el robot giraba muy rápido al detectar obstáculos, lo cual provocaba oscilaciones y giros bruscos, sobre todo en espacios estrechos. Decidimos reducir la velocidad angular a valores bajos (e.g. ± 0.1 rad/s) para lograr giros más suaves y estables. Además, limitamos la velocidad lineal en maniobras laterales, para evitar choques mientras se estaba girando.

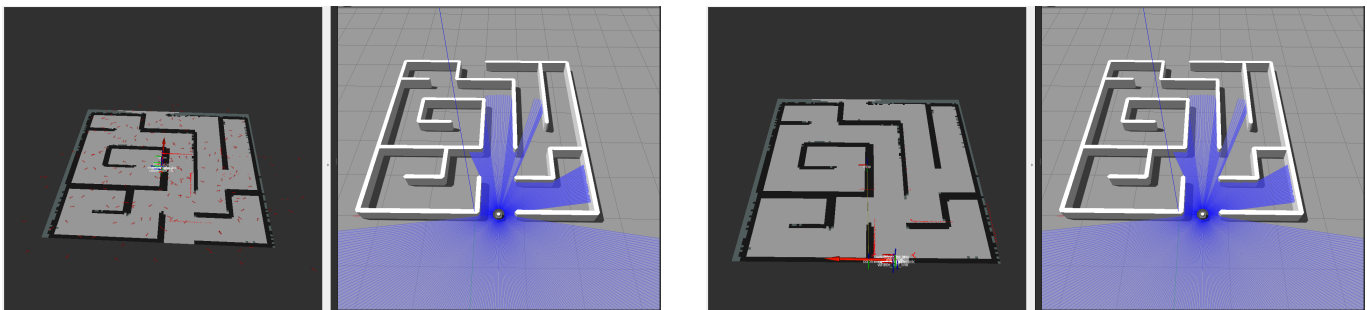
Gracias a estos ajustes, logramos que la evasión funcione de manera más robusta. El robot ya no se queda girando eternamente, ni vuelve a meterse en el obstáculo recién esquivado, y logra retomar su trayectoria sin movimientos erráticos. La simplicidad del algoritmo permitió mantener la evasión eficiente sin requerir mapas dinámicos ni cálculos costosos.

III-A5. Elección final de parámetros y criterios:

- num_particles: 200
- alpha1: 0.1
- alpha2: 0.1
- alpha3: 0.1
- alpha4: 0.1
- z_hit: 0.9
- z_rand: 0.1
- lookahead_distance: 0.6
- linear_velocity: 0.1
- goal_tolerance: 0.2
- path_pruning_distance: 0.1
- safety_margin_cells: 5

III-B. Resultados

III-B1. *Localización:* Uno de los primeros resultados obtenidos es la localización del robot. Como se anticipó anteriormente,



(a) Mala localización sin pose inicial, partículas por todo el mapa

(b) Mejor localización con pose inicial, partículas solo alrededor

el sampleo por todo el mapa no es bueno para estimar la pose del robot y necesita una pose inicial tentativa que esté alrededor de la pose real para que el robot pueda acomodarse.

A continuación se adjunta una Demostración en YouTube. donde se manejó el turtlebot con teleoperación (WASD) y vemos como el robot, a medida que transcurre el tiempo y se va moviendo, logra localizarse en el mapa de manera efectiva, tanto en posición como en orientación

III-B2. *Planeamiento:* Tal como se mencionó anteriormente, el path trazado por el algoritmo lleva el robot por el medio del pasillo, alejado de las paredes, gracias al inflado del mapa realizado anteriormente. Una idea que se pensó en su momento fue la de tomar el path trazado por el algoritmo y quitarle puntos para suavizarlo, porque tal como se ve en la imagen anterior, el trazo del camino no se ve del todo suave, sino más bien se ven pequeñas correcciones todo el tiempo. Para eso hicimos una

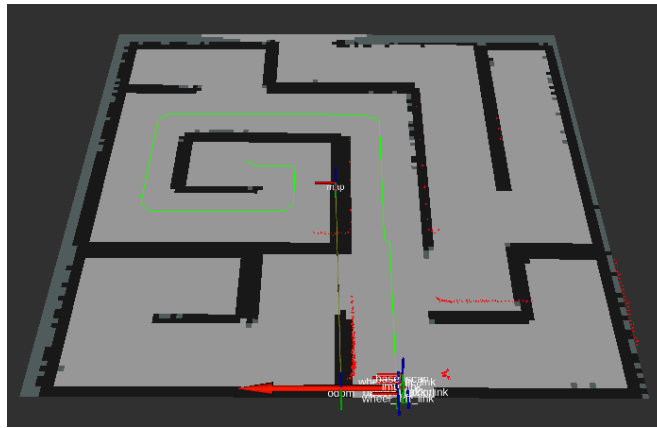


Figura 4: Path realizado por el algoritmo A^*

función y utilizamos el parámetro `path_pruning_distance`, donde pedimos que la distancia entre puntos del path sea mayor a 0.1 metros. Si bien el camino se suavizó, no se observaron diferencias significativas a la hora de seguir la trayectoria en los dos casos, por lo que se terminó eliminando esta idea, también con el objetivo de no introducir más parámetros que nos pudieran confundir a la hora de buscar la configuración óptima.

III-B3. Navegación y evasión: A través de la siguiente Demostración en YouTube es posible observar que no solo se le ha podido determinar un punto objetivo al robot y que ha sido capaz de llegar a él, sino que además pudo evadir un obstáculo que se le presentó en el camino. Además, seguido a evadir el obstáculo, el robot se detuvo por un momento. En un primer momento se consideró que fue un bug, pero en realidad, el algoritmo que hemos realizado para evadir obstáculos no solo considera elementos que vengan de frente, sino también de costado. El robot identificó una pared a su costado y giro levemente sobre su eje para evitarlo y seguir con la navegación. A su vez, es posible observar que el robot se detuvo antes de llegar a la posición exacta donde se esperaba que termine y eso es gracias al parámetro `goal_tolerance`, que evita que el robot se quede dando vueltas por la zona de llegada si ya está a una distancia leve del objetivo.